



CI1056 – ALGORITMOS E ESTRUTURA DE DADOS II

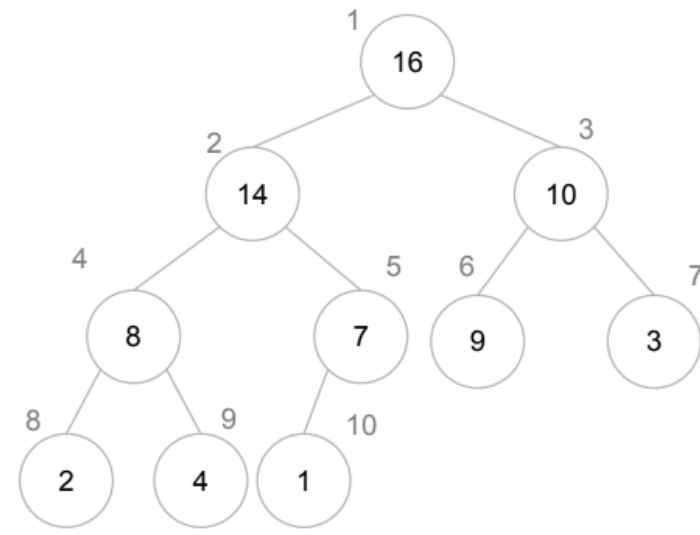
Profa. Rachel Reis
rachel@inf.ufpr.br

AULA DE HOJE

Algoritmo de Ordenação Heapsort

Adaptação do material gentilmente
disponibilizado pelos Profs. André Vignatti e
Marcos Castilho

HEAPSORT - MOTIVAÇÃO



- É um algoritmo de ordenação eficiente e elegante.
- Usa o *heap* de máximo que armazena o maior elemento na primeira posição do vetor.
- Vale lembrar que no *heap* de máximo os elementos não se encontram ordenados, pois não existe relação entre os elementos de um mesmo nível.

ORDENAÇÃO POR HEAP

- Problema computacional

Ordenação por *heap*

Instância: (v, n) , onde v é um vetor qualquer e n é o tamanho do vetor v $[1 .. n]$

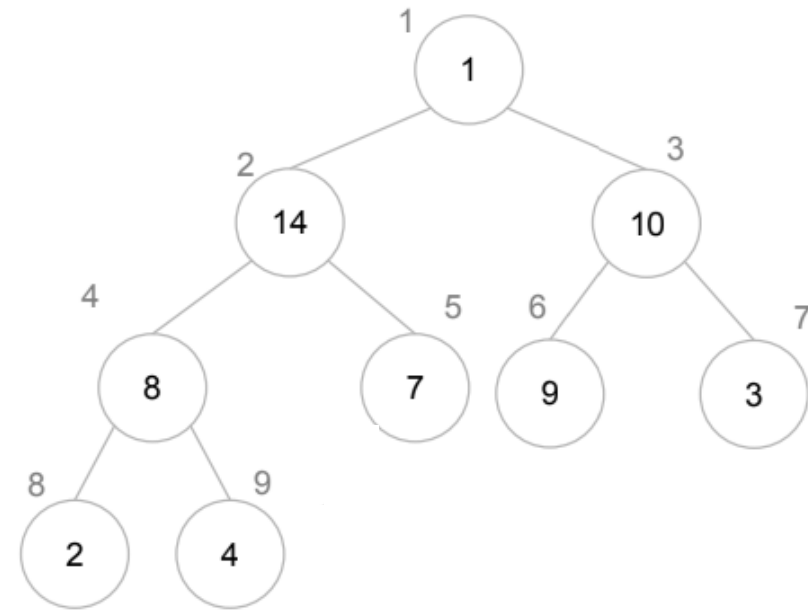
Resposta: vetor v modificado de forma que $v[1.. n]$ é um vetor ordenado.

Heapsort - Ideia

1	2	3	4	5	6	7	8	9	10
10	14	16	8	1	3	9	2	7	4

vetor qualquer

Heapsort - Ideia



1	2	3	4	5	6	7	8	9	10
1	14	10	8	7	9	3	2	4	16

- Primeiro constrói um heap a partir de um vetor qualquer.
- Em seguida, o maior elemento $h[1]$ é trocado com o último.
- O último elemento é desconsiderado, pois está na posição correta, e assumimos que o heap é $h[1.. n-1]$.
- Na sequência, chamamos o algoritmo *max-heapify*.
- Repetir o processo para $h[1.. n-1]$, $h[1.. n-2]$, etc.

HEAPSORT

- Algoritmo iterativo

heapsort(v, n)
build-max-heap(v, n) para i de n até 1 passo -1 Troca($v, 1, i$) max-heapify($v, 1$)

heapsort(v, n)

build-max-heap(v, n)
para i de n até 1 passo -1
Troca($v, 1, i$)
max-heapify($v, 1$)

EXECUÇÃO DO HEAPSORT

- Vetor de entrada (qualquer)

	1	2	3	4	5	6	7	8	9	10
v	15	35	40	10	20	5	45	70	25	90

- Com o custo linear (baseado no número de comparações) obtemos um *heap* de máximo.

	1	2	3	4	5	6	7	8	9	10
v	90	70	45	25	35	5	40	10	15	20

heapsort(v, n)

build-max-heap(v, n)
para i de n até 1 passo -1
Troca($v, 1, i$)
max-heapify($v, 1$)

EXECUÇÃO DO HEAPSORT

	1	2	3	4	5	6	7	8	9	10
v	5	10	15	20	25	35	40	45	70	90

Vetor final após a execução
do algoritmo de ordenação heapsort

ANÁLISE

heapsort(v, n)
build-max-heap(v, n) para i de n até 1 passo -1 Troca($v, 1, i$) max-heapify($v, 1$)

- Considere $\mathcal{C}(n)$, o número de comparações entre os elementos do vetor de tamanho n .

ANÁLISE

heapsort(v, n)
build-max-heap(v, n) para i de n até 1 passo -1 Troca($v, 1, i$) max-heapify($v, 1$)

- Algoritmo *build-max-heap* $C_{bmh}^+(n) = 4n$
- A cada iteração do algoritmo *max-heapify*

$$C_{mh}^+(n) = 2 \cdot \lfloor \log_2 n \rfloor \approx 2 \cdot \log_2 n$$

- Na primeira iteração: $2 \cdot \lfloor \log_2(n - 1) \rfloor$
- Na segunda iteração: $2 \cdot \lfloor \log_2(n - 2) \rfloor$
- Na terceira iteração: $2 \cdot \lfloor \log_2(n - 3) \rfloor$
- ...
- Na última iteração: $2 \cdot \log_2(1)$

ANÁLISE

- Logo:

$$\begin{aligned} C(n) &= 4n + 2 \cdot \lfloor \log_2(n-1) \rfloor + 2 \cdot \lfloor \log_2(n-2) \rfloor + \dots + 2 \cdot \log_2(1) \\ &\leq 4n + 2(\log_2(n-1) + \log_2(n-2) + \dots + \log_2(1)) \\ &= 4n + 2(\sum_{i=1}^{n-1} \log_2(n-i)) \\ &= 4n + 2(\sum_{i=1}^{n-1} \log_2 i) \\ &\leq 4n + 2(\sum_{i=1}^n \log_2 i) \\ &\leq 4n + 2(\sum_{i=1}^n \log_2 n) \\ &= 4n + 2(n \cdot \log_2 n) \end{aligned}$$

Foi adicionado um elemento
ao somatório (n)

$\log 1 + \log 2 + \dots + \log n$
 $\leq \log n + \log n + \dots + \log n$

ANÁLISE

- Logo:

$$C(n) = 4n + 2 \cdot \lfloor \log_2(n-1) \rfloor + 2 \cdot \lfloor \log_2(n-2) \rfloor + \dots + 2 \cdot \log_2(1)$$

- Isso resulta em um custo aproximado:

$$C(n) \leq 4n + 2(n \cdot \log_2 n)$$

CONCLUSÃO

- Algoritmos de ordenação “ingênuos”:
 - Selection sort: $C(n) \approx n^2$
 - Insertion sort: $\begin{cases} C^-(n) \approx n \\ C^+(n) \approx n^2 \end{cases}$

CONCLUSÃO

- Algoritmos de ordenação “espertos”:
 - Heapsort: $C(n) = n \log_2(n)$
 - Quicksort: $\begin{cases} C^-(n) = n \log_2 n \\ C^+(n) \approx n^2 \end{cases}$
 - Mergesort: $C(n) = n \log_2 n$